

Instructions for Lab Submission

1. File Naming and Submission:

- Submit your solution as a **ZIP file** named `lab1.<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
- Ensure that the ZIP file contains:
 - C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
 - No unnecessary files (such as executables, temporary files, or editor configs).
- Do not submit **TEXT(.txt)** files.

2. Code Organization:

- Write each task in its own `.cpp` file (as specified above).
- Ensure that your code is modular and logically separated into functions when specified.

3. Code Neatness:

- Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
- Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
- Add clear explanations where required (or specified)** to make the logic of your program easy to follow.

4. Code Compilation and Functionality:

- Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
- Double-check that the functionality of the program meets the requirements outlined in the task.

5. Additional Instructions:

- Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
- Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
- If you encounter any issues or require clarifications, reach out before the submission deadline.

6. Deadline:

- Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
- Double-check that the zip file is correctly named and contains all necessary files before submitting.

7. Plagiarism Policy:

- The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

Hello World!

Recall the following simple program from first lecture that demonstrates the basic structure of a C++ program, including libraries, defining the main function, and using the `cout` object for output.

```
#include <iostream>
int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

Now, let's break down the different parts of this program to make it understandable:

- `#include <iostream>`: This line is a preprocessor directive that tells the compiler to include the `iostream` library. The `iostream` library provides functionality for input and output operations, like printing text to the screen.
- `int main() ...`: This is the main function of your program. Every C++ program must have a main function. The program's execution starts from here.
- `std::cout << "Hello, World!" << std::endl;`: This line uses the `cout` object from the `iostream` library to print text to the console. `std::cout` is used to output data to the console, and `<<` is the insertion operator that inserts the text or value on its right into the output stream. "Hello, World!" is the text you want to print. `std::endl` is used to insert a newline character and flush the output buffer. You can also use `'\n'` character inside the string for a new line, e.g., `std::cout << "Hello, World!\n"`.
- `return 0;`: This line indicates the end of the main function and the successful termination of the program. The value 0 is returned to the operating system, signifying that the program ran without errors.

Setting up g++ Compiler

Before running this program, you need to compile it using a C++ compiler like `g++`. Verify that you have a C++ compiler installed by running the command:

```
g++ --version
```

If `g++` is not recognized as a command, you need to install `g++` compiler and/or add the path to the compiler to the `PATH` environment variable.

- **On your laptop or home computer (Windows):** Follow the guide at <https://www.msys2.org/> to install MSYS2 and set up the `g++` compiler on your system.
- **In labs:** The C++ compiler is already installed on the lab computers. You may need to set the `PATH` environment variable to run the compiler from the command line.

Open Windows Command Prompt by pressing Win + R and typing `cmd`. Then run the following command in the terminal:

```
set PATH=C:\msys64\ucrt64\bin;%PATH%
```

Compiling and Running a C++ Program in VS Code

Open VS Code → Open a Folder → Make a new file → Give it any name just end it with `.cpp`. After writing the code press Ctrl+Shift+P and click open new terminal. In the terminal write the following command; this command can vary if the file name is different. Make sure you save the code with Ctrl+S before running the command otherwise it will cause an error.

```
g++ hello.cpp -o hello.exe
./hello.exe
```

where `hello.cpp` is the name of your program file, and `hello.exe` is the name of the executable file generated by the compiler.

If there is an error in your program, the compiler will display an error message indicating the line number and the type of error. For example, if you forget to put a semicolon after the `cout` statement, you'll get an error like:

```
error: expected ';;' before 'return'
```

When you compile and run this program, you'll see the output: `Hello, World!`

Declaration and Assignment Statements

In C++, we use variables to store data. A variable is a named location in memory that stores a value. Unlike Python or JavaScript, C++ is a statically typed language, which means that we need to specify the data type of a variable when we declare it. Some of the common data types in C++ are `int` (integer), `double` (floating-point number), and `std::string` (string). Here's how we declare variables of different types:

```
int age;  
double price;  
std::string name;
```

To assign a value to a variable, you use the assignment operator `=`. For example, you can use variables to store a person's age, a product's price, or a student's grade.

```
age = 25;  
price = 19.99;  
name = "Alice";
```

A declaration and assignment statement can be combined into a single line:

```
int age = 25;  
double price = 19.99;  
std::string name = "Alice";
```

Input/Output using `cin` and `cout`

`std::cin` and `std::cout` are part of the C++ Standard Library and are used for input and output operations. `cin` is used for input (reading values from the user), and `cout` is used for output (displaying values to the user).

Input using `cin`: To get input from the user, you use `cin` along with the `>>` operator. You need to specify the variable where the input should be stored. Here's an example of how to read an integer from the user and store it in the `age` variable:

```
cin >> age;
```

Output using `cout`: To display output to the user, you use `cout` along with the `<<` operator. You can output variables, constants, and text. Here's an example:

```
std::cout << "Your age is: " << age << std::endl;
```

Escape Sequences

In C++, you can use escape sequences to represent special characters in strings.

Escape Sequence	Description
<code>\n</code>	newline
<code>\t</code>	tab
<code>\r</code>	carriage return (it means “go back to the start of the same line and write again from there.”)
<code>\b</code>	backspace
<code>\\</code>	backslash
<code>\"</code>	double quote
<code>\'</code>	single quote

For example, the following code will output the text on two lines:

```
std::cout << "Hello, World!\n";
std::cout << "Welcome to C++ Programming\n";
```

The above lecture review is based on the ITP Lab #1 material.

Data types

Recall that a *data type* is set of values and set of operations on those values. It determines the possible values that can be stored in a variable and the operations that can be performed on that variable. It also determines the amount of memory required to store the value and the way the value is stored in the memory. Some commonly used data types in C++ are following:

- **int** - stores integers (whole numbers), without decimals, such as 123 or -123. The **int** data type stores values that are within a range of -2,147,483,648 to 2,147,483,647 (32-bit signed integer).
Other useful data types for integers are: **short** (stores 16-bit integers), **long** (stores 32-bit integers), **long long** (stores 64-bit integers).
- **double** - stores floating point numbers, with decimals, such as 19.99 or -19.99. The **double** data type is a double-precision 64-bit floating point.
Other useful data types for floating point numbers are: **float** (stores 32-bit floating point numbers), **long double** (stores 128-bit floating point numbers).
- **bool** - stores values with two states: **true** or **false**
- **char** - stores single characters, such as 'a' or 'B'. The **char** values are surrounded by single quotes.
The **char** is a type of integer, so it also stores numbers, which are interpreted as characters according to the ASCII encoding. For example, the ASCII value for the character 'a' is 97 and the ASCII value for the character 'b' is 98.
- **std::string** - stores text, such as "Hello World". The **std::string** values are surrounded by double quotes.

Some functions from the C++ library needed for the lab

function	description	example	library
<code>min()</code>	returns the minimum of two numbers	<code>int m = std::min(3, 5);</code>	<code><algorithm></code>
<code>max()</code>	returns the maximum of two numbers	<code>std::cout << std::max(3, 5);</code>	<code><algorithm></code>
<code>abs()</code>	returns the absolute value of a number	<code>int x = std::abs(-10);</code>	<code><cmath></code>
<code>pow()</code>	returns the first argument raised to the power of the second argument	<code>double y = std::pow(2, 3);</code>	<code><cmath></code>
<code>sqrt()</code>	returns the square root of a number	<code>double s = std::sqrt(16);</code>	<code><cmath></code>
<code>ceil()</code>	returns the smallest integer that is greater than or equal to the argument	<code>int z = std::ceil(9.2);</code>	<code><cmath></code>
<code>floor()</code>	returns the largest integer that is less than or equal to the argument	<code>int z = std::floor(9.2);</code>	<code><cmath></code>
<code>lround()</code>	returns the long integer that is closest to the argument	<code>long z = std::lround(9.2);</code>	<code><cmath></code>
<code>time()</code>	returns the current time of the system as time since epoch	<code>std::cout << std::time(0);</code>	<code><ctime></code>
<code>rand()</code>	returns a random number	<code>int r = std::rand();</code>	<code><cstdlib></code>
<code>srand()</code>	sets the seed for the random number generator	<code>std::srand(std::time(0));</code>	<code><cstdlib></code>

You need to add appropriate library file to be able to use a function. For instance:

```
#include <iostream>
#include <cmath>
int main() {
    std::cout << "Square root of 2 is: "
              << std::sqrt(2);
}
```

Lab Tasks

Task 1: Secret Message Display

Write a C++ program that displays a short secret message using comments and escape sequences. The message should contain quotation marks around **Mission Complete**.

Requirements

- Use at least one single-line comment using `//`.
- Use at least one multi-line comment using `/* */`.
- Use `\n` for new lines.
- Use `\t` for spacing between labels and values.
- Use `\"` to display quotation marks.
- Use **only one** `cout` statement for the complete message.
- Do not take input from the user.
- Do not use variables, calculations, conditions, or loops.

Output

```
=====
SECRET MESSAGE
=====
Status: "Mission Complete"
Message:    Keep Learning!
=====
```

Figure 1: Task 1 Output

Task 2: Transforming Words Using Carriage Return

Write a C++ program that demonstrates the use of the carriage return escape sequence `\r` to overwrite characters in a previously printed word.

Problem Description

The program should initially print the word **CAT**. Using `\r`, move the cursor back to the beginning of the line and replace the first two characters with **SI**, transforming the word from **CAT** to **SIT**.

Then, use `\r` again and replace the first two characters with **PI**, transforming **SIT** into **PIT**.

Output Workflow

Follow the given sequence:

1. Print `CAT`.

2. Use `\r` to move the cursor to the beginning of the line.
3. Print `SI` to replace the first two characters of `CAT`, resulting in `SIT`.
4. Use `\r` again to move the cursor to the beginning of the line.
5. Print `PI` to replace the first two characters of `SIT`, resulting in `PIT`.

Requirements

- Use the carriage return escape sequence `\r`.
- Do not use backspace characters.
- Use `CAT`, `SI`, and `PI` in the specified order.
- Demonstrate how `\r` can be used to overwrite only part of previously printed output.
- The final displayed word should be **PIT**.

Test Case

```
Output :
PIT
```

Output



Figure 2: Task 2 Output

Task 3: Student Introduction Card

Write a C++ program that asks the user for their name and displays a personalized student introduction card.

Requirements

- Declare a variable of type `string` named `name`.
- Ask the user to enter their name using `cin`.
- Display the entered name using `cout`.
- Use the variable instead of writing the name directly in the output.
- Use only one input value.
- Use `\n` to display the card on multiple lines.
- Include the required library for string input.
- Do not use calculations, conditions, or loops.

Output

```
Enter your name: Ali

-----
                STUDENT INTRODUCTION
-----

Welcome, Ali!
You are now part of the class.
-----
```

Figure 3: Task 3 Output

Task 4: Grocery Store Receipt.....

You have been assigned as a junior developer at a retail store. Your first task is to develop a simple C++ program that displays a grocery store receipt on the screen.

The receipt contains fixed information about a customer's purchase. No input is required from the user. Your program should simply display the given information in a neat and well-aligned receipt format.

Problem Description

Create a C++ program that displays a grocery store receipt containing the following information:

- Store name
- Customer name
- Grocery item
- Quantity purchased
- Price per item
- Payment method

All the information should be directly written in the program and displayed when the program is executed.

Requirements

- Display all the required information using `cout`, data can be of your choice.
- Use escape sequences such as `\n` and `\t` to format the receipt neatly.
- The output should be properly aligned and easy to read.
- Use only one `cout` statement to display the entire receipt.

Main Function Instructions

- Write the receipt information directly inside the program.
- Use a single `cout` statement to print the complete receipt.

- Use `\n` to move to a new line and `\t` to create appropriate spacing between labels and their values.
- Make the final output resemble a simple grocery store receipt.

Test Case

```
Output:
=====
                GROCERY RECEIPT
=====
Store:           Fresh Mart
Customer:        Abdullah
Item:            Milk
Quantity:        3
Price/Item:      250
Payment:         Cash
=====
                Thank you for shopping!
=====
```

Output

```
=====
                GROCERY RECEIPT
=====
Store:           Fresh Mart
Customer:        Abdullah
Item:            Milk
Quantity:        3
Price/Item:      250
Payment:         Cash
=====
                Thank you for shopping!
=====
```

Figure 4: Task 4 Output

Task 5: Introduction to Expressions

Before learning variables and expressions in detail, let us take a first look at how C++ can perform simple calculations directly inside a `cout` statement.

Quick Demo

C++ can evaluate a mathematical expression and display its result:

```
#include <iostream>
using namespace std;

int main() {
    cout << 5 + 3;
```

```
}    return 0;  
}
```

Output:

```
8
```

Here, `5 + 3` is an **expression**. C++ evaluates the expression first and then displays its result.

Some other examples of simple expressions are:

```
cout << 10 * 2;  
cout << 20 / 4;
```

You do not need to use variables for this task.

Task

Write a C++ program that displays the result of adding **25** and **15**.

The program should use an expression directly inside `cout`.

Requirements

- Use `cout` to display the result.
- Perform the addition directly inside the `cout` statement.
- Do not use variables.
- Do not take input from the user.

Test Case

```
Output :  
  
40
```

Output



25 + 15 = 40

Figure 5: Task 5 Output